PROOFENGINE V0.1.0 — OPERATOR GUIDE

How to Run the ProofEngine Demo & Live Demo

Deterministic Symbolic Execution & Z3 Verification Layer for Financial, Billing, and Legal Code

1. Prerequisites & Workspace Setup

Open your terminal and navigate to the ProofEngine repository root:

```
cd /Users/devanshsuri/Documents/trae_projects/ProofEngine
```

Package	Location	Responsibility
`@proofengine/core`	`packages/core`	Babel + ts-morph AST parser, `@proofengine` DSL extractor & SMT-LIB2 generator
`@proofengine/solver-api`	`packages/solver-api`	Z3 SMT solver runner (5s timeout), HS256 JWT proof signer & 10-call quota store
`@proofengine/cli`	`packages/cli`	Developer CLI (`proofengine verify` , `check` , `quota` , `subscribe`)
`@proofengine/mcp-server`	`packages/mcp-server`	Model Context Protocol (MCP) stdio server for Cursor / Windsurf / Claude agents

2. Part A — How to Run the Automated Demo

Demo 1: Run All 5 Milestone Verification Suites

Executes AST/SMT generation tests, Z3 solver proof/counterexample tests, MCP stdio server tests, and the 10-verification freemium quota gate:

```
pnpm test
# Or run individual milestone suites:
pnpm test:core     # Milestone 1: AST extraction & SMT-LIB2 generation
pnpm test:solver   # Milestone 2: Z3 UNSAT JWT proof & SAT counterexample (<3s)
pnpm test:mcp      # Milestones 4 & 5: MCP stdio tools & HTTP 402 payment gate
```

Demo 2: Catch the Stripe Proration Rounding Drift Bug (Exits with Code 1)

Run the CLI against `test/fixtures/proration.ts`, which floors per-seat amounts before multiplying by seats (causing a 33.3% / fractional-cent rounding drift):

```
npx proofengine verify test/fixtures/proration.ts --no-quota
```

```
  ProofEngine  v0.1.0 — Symbolic Verification
  ──────────────────────────────────────────────────
  Target:  .../test/fixtures/proration.ts
  Parsed:  1 invariants · 3 boundaries · 0 control branches · 7 arithmetic nodes

   1. ✖ VIOLATION (SAT)  billed_cents == exact_proration_cents
       vars:8 · constraints:11 · 199ms

     Counterexample (satisfying assignment):
       days                          29
       seats                          2
       pricePerSeat                 100
       exactProrationCents        193.3333
       perSeatCents                  96
       billedCents                  192
       drift                     -1.3333  ⚠ drift=-1.3333

     Counterexample JSON trace: {"inputs":{"days":29,"seats":2,"pricePerSeat":100},...
```

Demo 3: Verify the Fixed Stripe Proration Calculation (Exits with Code 0)

Run the CLI against `test/fixtures/proration-fixed.ts`, where floor rounding is applied on the aggregate total:

```
npx proofengine verify test/fixtures/proration-fixed.ts --no-quota
```

```
   1. ✔ PROVEN (UNSAT)  billed_cents == exact_proration_cents
       vars:7 · constraints:10 · 141ms · proof:eyJhbGciOiJIUzI1...

  ──────────────────────────────────────────────────
  Summary: 1 proven · 0 violations · 161ms
```

3. Part B — How to Run the Interactive Live Demo

Follow this 4-step live sequence during a presentation to demonstrate catching a bug in real time and mathematically proving the fix.

Step 1: Open `examples/live-demo.ts` and Introduce an Intentional Bug

Edit `examples/live-demo.ts` so that `total` subtracts `1` instead of adding the fee, violating `total >= amount`:

```
// @proofengine:boundary amount >= 0
export function calculateCheckoutTotal(amount: number): number {
  // Intentional bug: subtracting 1 violates total >= amount
  // @proofengine:invariant total >= amount
  const total = amount - 1;
  return total;
}
```

Step 2: Run Verification Live — Watch Z3 Catch the Bug

```
npx proofengine verify examples/live-demo.ts --no-quota
```

```
  1. ✖ VIOLATION (SAT)  total >= amount
       vars:2 · constraints:3 · 159ms

    Counterexample (satisfying assignment):
      amount                          0
      total                          -1  ⚠ drift=-1

    Counterexample JSON trace: {"inputs":{"amount":0},"computed":{"total":-1},"drift"
```

Step 3: Fix the Code Live and Re-Verify

Change `const total = amount - 1;` to `const total = amount + 1;` in `examples/live-demo.ts` and re-run:

```
npx proofengine verify examples/live-demo.ts --no-quota
```

```
  1. ✔ PROVEN (UNSAT)  total >= amount
       vars:2 · constraints:3 · 168ms · proof:eyJhbGciOiJIUzI1...

  ──────────────────────────────────────────────────
  Summary: 1 proven · 0 violations · 184ms
```

Step 4: Show SMT-LIB2 Equations, JSON CI Output, Linter & Quota Gate

```
# 1. Print the generated SMT-LIB2 mathematical program sent to Z3:
npx proofengine verify examples/live-demo.ts --no-quota --debug-smt

# 2. Print machine-readable JSON output (with signed HS256 JWT proof):
npx proofengine verify examples/live-demo.ts --no-quota --json

# 3. Scan the codebase for untagged payment/billing keywords:
npx proofengine check examples

# 4. Demo the 10-verification freemium quota & Developer Pro ($49/mo) upgrade:
npx proofengine quota --reset
npx proofengine verify examples/live-demo.ts
npx proofengine quota
npx proofengine subscribe dev-pro --auto-approve
```

4. Part C — Live AI Agent Demo (Cursor / MCP Server)

The workspace includes `.cursor/mcp.json` and `.cursor/rules/proofengine.mdc` (matching `**/billing/**`, `**/stripe/**`, `**/payments/**`, `**/subscriptions/**`, `**/tax/**`). In Cursor or any MCP client, ask the agent:

```
"Verify examples/billing-buggy.ts using verify_financial_invariants and fix any counterexam
```

The agent will invoke the `verify_financial_invariants` MCP tool, read the Z3 counterexample trace, patch the code, and re-run verification until it receives a signed `PROVEN` JWT token.